BIG DATA · Grado en Ciencia e Ingeniería de Datos · ULPGC · 2026–2027

Assignment 1: Basic Matrix Multiplication in C, Java and Python

How does matrix size affect the runtime and memory use of one triple-loop algorithm in three languages?

Amado Artiles Rodríguez

October 2026

Repository: https://github.com/amadoartiiles/matrix-multiplication-studies

Abstract

This study measures how the matrix size n affects the runtime and peak memory of the same $O(n^3)$ triple-loop multiplication of binary64 matrices in C, Java and Python, using identical inputs, the same i-j-k loop order and the same timing boundary on one laptop (AMD Ryzen 7 7730U, Windows 11), for $n = 32$ to 2048 with five measured repetitions each. All implementations gave bitwise-identical results within a rigorous error bound of a NumPy reference. CPython was 17–130× slower than C, exceeded the 300 s budget at $n = 1536$ and grew close to n^3 (fitted exponent 3.14). C and Java stayed within a factor of two of each other but grew faster than n^3 (3.50 and 3.73) as the matrices left the caches; padding each C row by 16 bytes made the same code 1.9–6.9× faster, pointing to a memory-layout effect. Peak memory was dominated by the runtime baseline for small n and by the matrices for large n. The conclusions apply to these implementations, settings and machine only.

Keywords: matrix multiplication, benchmarking, C, Java, Python, cache effects, memory layout.

1 Introduction and objective

Dense matrix multiplication is the reference kernel of numerical computing and a common first benchmark when comparing execution environments. Its textbook form performs exactly n^3 multiply-add operations for $n \times n$ inputs, so differences in runtime between implementations of the *same* algorithm expose the cost of the execution environment itself: interpretation, JIT compilation, data representation and memory layout.

Research question

For the straightforward triple-loop algorithm, how does the matrix size n affect the runtime and the peak memory of implementations in C, Java and Python on one fixed machine, and how does the observed growth compare with the cubic model?

Hypotheses (stated before measuring)

H1 All three runtimes grow approximately as n^3 once fixed start-up costs are excluded.

H2 The compiled C version is fastest, Java is within a small factor of C after JIT warm-up, and CPython is one to two orders of magnitude slower because every inner iteration is interpreted.

H3 Memory grows as n^2; Python needs several times more memory per element than C and Java because each value is a separate heap object.

2 Background

Algorithm

$C_{ij} = \sum_{k=0}^{n-1} A_{ik} B_{kj}$ evaluated with three nested loops performs n^3 multiplications and n^3 additions, i.e. $2n^3$ floating-point operations, and is $\Theta(n^3)$ independent of the language [1]. In the i-j-k order the innermost loop reads a row of A with unit stride and a *column* of B with a stride of one row, which is the cache-unfriendly access pattern that blocking and loop reordering later address [5, 6].

Floating point

All three languages use IEEE 754 binary64 arithmetic [3]. Java has had strict IEEE semantics for all floating-point operations since Java 17 [4]; C compiled without `-ffast-math` may not reassociate the sum; CPython's `float` is a C `double`. Evaluating the same sums in the same order should therefore give identical results.

For a dot product of length n, the computed product satisfies, element-wise,

$$|\hat{C} - C| \le \gamma_n |A||B|, \qquad \gamma_n = \frac{nu}{1 - nu}, \qquad u = 2^{-53} \quad [2, \text{Sec. } 3.5].$$

Execution environments

C is compiled ahead of time to machine code. Java bytecode is first interpreted and then compiled by the HotSpot JIT compiler, which is why warm-up runs are recommended before measuring [10]. CPython interprets bytecode and represents each number as a boxed heap object (24 bytes for a `float`, plus an 8-byte pointer in the list that holds it).

Caches

The Zen 3 core of the test CPU has a 32 KB 8-way L1 data cache and a 512 KB 8-way L2 cache per core, plus a 16 MB L3 shared by eight cores [8]. When the distance between consecutive accessed addresses is a large power of two, they map to only a few cache sets and evict each other even though the cache is not full (conflict misses) [5, 7].

3 Methodology

3.1 Implementations

The kernel is the same in the three languages (Figure 1). It allocates and zero-initialises the result, then runs the i-j-k loop with a local accumulator `s`. No library multiplication routine is called and no language-specific optimisation (loop hoisting, unrolling, transposition) was written by hand.

C · c/matmul.c

```c
static double *multiply(const double *A, const double *B, int n) {
    double *C = (double *)calloc((size_t)n * n, sizeof(double));
    if (C == NULL) return NULL;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            double s = 0.0;
            for (int k = 0; k < n; k++) {
                s += A[(size_t)i * n + k] * B[(size_t)k * n + j];
            }
            C[(size_t)i * n + j] = s;
        }
    }
    return C;
}
```

Java · java/src/MatMul.java

```java
static double[][] multiply(double[][] A, double[][] B, int n) {
    double[][] C = new double[n][n];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            double s = 0.0;
            for (int k = 0; k < n; k++) {
                s += A[i][k] * B[k][j];
            }
            C[i][j] = s;
        }
    }
    return C;
}
```

Python · python/matmul.py

```python
def multiply(A, B, n):
    C = [[0.0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            s = 0.0
            for k in range(n):
                s += A[i][k] * B[k][j]
            C[i][j] = s
    return C
```

Figure 1: The timed kernels, exactly as in the repository (the timed region is the whole function).

Storage layouts (unavoidable differences)

- **C:** one contiguous row-major `double` buffer per matrix ($8n^2$ bytes), allocated with `calloc`.
- **Java:** `double[n][n]`, an array of n references to separately allocated row arrays; each row has a 16-byte object header, so consecutive rows are normally $8n + 16$ bytes apart on the heap.
- **Python:** a list of n row lists, each holding n pointers to boxed `float` objects.

The analytical storage of A, B and C ("storage model" below) is therefore $3 \cdot 8n^2$ bytes in C,

$\approx 3(8n^2 + 16n)$ in Java and $\approx 3(32n^2 + 56n)$ in Python.

3.2 Inputs

`scripts/generate_inputs.py` creates A and B for every n with NumPy's PCG64 generator seeded with `[20261012, n]`, values uniform in $[-1, 1)$, and writes them as little-endian binary64 row-major files with SHA-256 hashes. All three programs read the same files, so the logical inputs are identical across languages.

3.3 Environment and fixed settings

The settings in Table 1 were identical for every size. The file `results/environment.json` records the exact environment.

Table 1: Test environment and fixed settings.

CPU	AMD Ryzen 7 7730U (Zen 3), 8 physical cores / 16 logical processors
Memory, OS	16 GB RAM; Windows 11 Home 10.0.26200; HP Laptop 15-fc0xxx; on AC power, "Balanced" power plan
C	GCC 16.2.0 (MinGW-w64 UCRT); flags `-O2 -std=c11 -Wall -Wextra` (no `-march`, no fast-math)
Java	Oracle JDK 24.0.1, HotSpot Server VM, G1 GC, `-Xmx4g`; 3 warm-up runs
Python	CPython 3.11.9, no flags, standard library only; 1 warm-up run
Threads	all kernels are single-threaded; no numerical library is used inside the timed code

3.4 Measurement protocol

Timing boundary

Kernel time only: allocation and zero-initialisation of C plus the triple loop. Reading the inputs, checksums and output files are outside the timed region. Monotonic timers: `QueryPerformanceCounter` (C), `System.nanoTime` (Java), `time.perf_counter_ns` (Python).

Repetitions

Each (language, n) configuration runs in a fresh process: warm-up runs (1 for C and Python, 3 for Java) followed by five measured repetitions. If one multiplication took less than 0.1 s, each measured sample is a batch of $\lceil 0.1/t_{\text{warm-up}} \rceil$ consecutive multiplications and the reported value is the batch time divided by the batch size.

Every result is consumed (a `volatile` sink in C and Java, and a checksum of the last result) so that no compiler can remove the work. We report the median and the interquartile range (IQR) of the five samples; all raw samples, including warm-ups, are kept in `results/raw/measurements.csv`. Configurations ran in the order n ascending and, for each n, C, Java, Python, with a 2 s pause between processes.

Memory

The driver samples the child process every 20 ms with `psutil` and records the Windows *peak working set* (`PeakWorkingSetSize`, an OS-maintained high-water mark of resident memory) [11]. This is a whole-process metric and includes the runtime itself (JVM, interpreter). For Java we also record the managed heap in use after the last repetition (`MemoryMXBean`). These metrics are reported separately and not mixed.

Budgets declared before the large runs

300 s per single multiplication and 4 GB peak working set per process. A configuration exceeding a budget is stopped and recorded with its limit; larger sizes of that language are then not attempted.

Sizes: 32, 64, 128, 256, 384, 512, 768, 1024, 1536, 2048 (powers of two plus intermediate sizes to show whether trends are smooth).

3.5 Correctness checks

Before benchmarking

Every implementation passes known-answer tests: a hand-computed 3×3 product, $AI = A$, $IA = A$, $A \cdot 0 = 0$, a non-commuting 2×2 case and a 1×1 case (exact comparison, since all values are small integers); see `results/selftest.txt`.

After benchmarking

The result of the last repetition of every configuration is compared with NumPy's `A @ B` (used only as a reference), element by element, using the bound

$$|C_{\text{impl}} - C_{\text{ref}}| \leq 2\gamma_n(|A||B|),$$

because both computations satisfy the error bound of Section 2 with respect to the exact product.

This bound is the tolerance, and it plays the role of both an absolute and a relative tolerance. For each element it is an absolute limit on the difference, but that limit is relative to the corresponding element of $|A||B|$: the factor $2\gamma_n$ ranges from 7×10^{-15} at $n = 32$ to 4.5×10^{-13} at $n = 2048$. Scaling by $|A||B|$ rather than by $|C_{ij}|$ is appropriate because cancellation in the sum can make C_{ij} close to zero, where a pure relative tolerance would reject correct results.

4 Results

4.1 Correctness

All 28 completed configurations satisfy the tolerance; the largest error was at most 3.4 % of the bound (`results/validation.csv`). At every size, the C, Java and Python results are *bitwise identical* to each other, and the checksums of all repetitions agree.

The three implementations therefore perform the same arithmetic, which is a precondition for comparing their runtimes.

4.2 Runtime

Figures 2–4 and Tables 2–3 summarise the runtime measurements. Table 2 gives the growth exponent fitted to the medians.

Table 2: Least-squares fit of $\log t = \log c + b \log n$ to the medians for $n \geq 128$.

Language	Sizes used	Fitted exponent b
C	128–2048 (8 sizes)	3.50
Java	128–2048 (8 sizes)	3.73
Python	128–1024 (6 sizes)	3.14

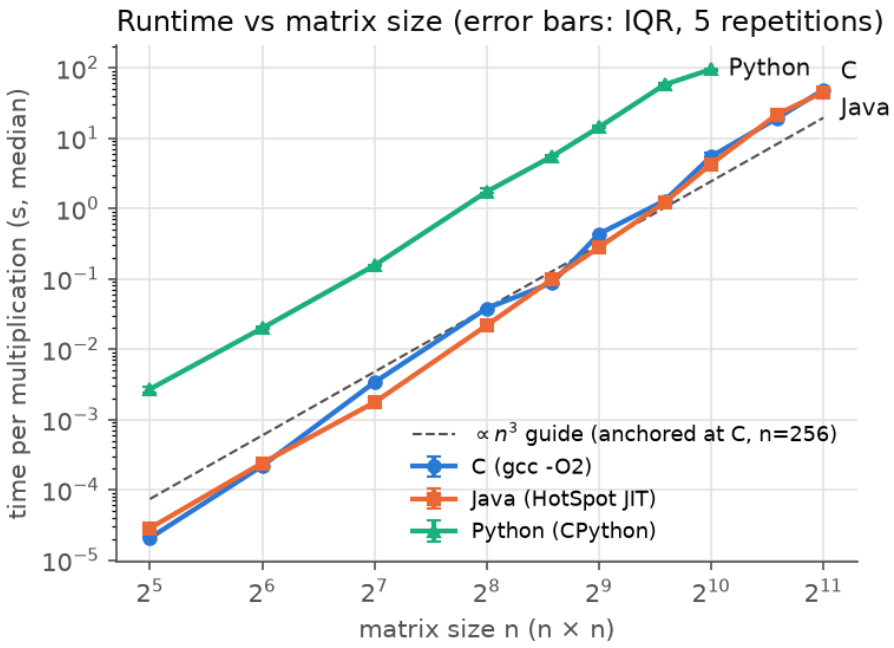


Figure 2: Median time per multiplication (log-log) with IQR error bars (5 repetitions; most bars are smaller than the markers). The dashed line has slope 3. Python stops at $n = 1024$ because $n = 1536$ exceeded the 300 s budget.

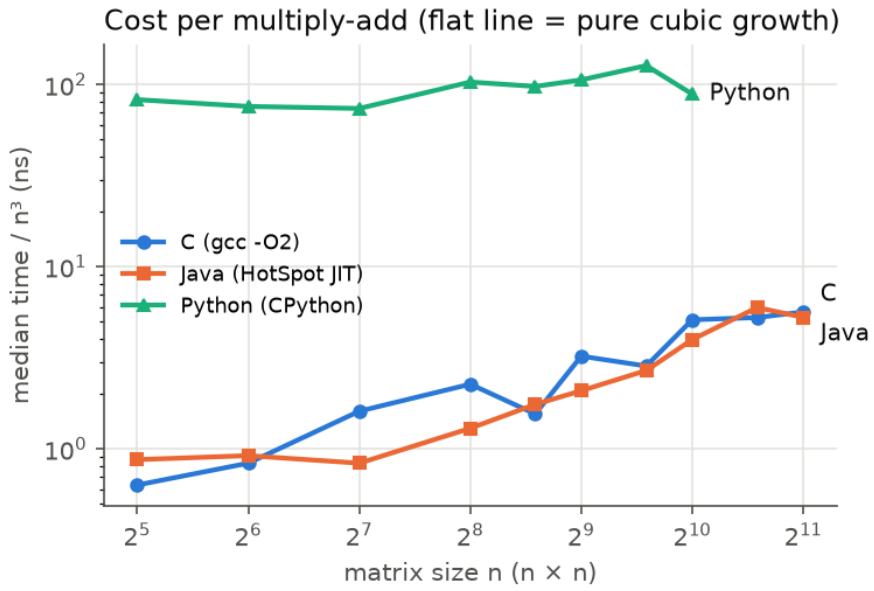


Figure 3: Median time divided by n^3. A horizontal line would mean exactly cubic growth.

Table 3: Runtime per multiplication: median over 5 measured repetitions, relative IQR (IQR/median), median time divided by n^3 (ns per inner iteration), $2n^3$/median in GFLOP/s, and batch size (multiplications per sample).

n	Language	Median	IQR (%)	ns/iter	GFLOP/s	Batch
32	C	20.7 μs	7.9	0.63	3.162	3509
32	Java	28.6 μs	1.8	0.87	2.295	334
32	Python	2.71 ms	19.9	82.58	0.024	29
64	C	218.5 μs	11.2	0.83	2.400	496
64	Java	240.4 μs	3.9	0.92	2.181	254
64	Python	19.87 ms	9.1	75.79	0.026	6
128	C	3.37 ms	1.4	1.61	1.244	30
128	Java	1.75 ms	6.1	0.83	2.400	51
128	Python	154.92 ms	4.8	73.87	0.027	1
256	C	37.96 ms	0.8	2.26	0.884	3
256	Java	21.74 ms	7.2	1.30	1.544	5
256	Python	1.73 s	17.4	103.18	0.019	1
384	C	87.92 ms	1.3	1.55	1.288	2
384	Java	99.18 ms	10.7	1.75	1.142	2
384	Python	5.53 s	2.9	97.60	0.020	1
512	C	432.93 ms	0.6	3.23	0.620	1
512	Java	279.73 ms	15.4	2.08	0.960	1
512	Python	14.23 s	9.2	106.03	0.019	1
768	C	1.29 s	3.3	2.85	0.703	1
768	Java	1.22 s	31.7	2.69	0.744	1
768	Python	57.52 s	8.9	126.99	0.016	1
1024	C	5.48 s	12.5	5.11	0.392	1
1024	Java	4.27 s	0.7	3.98	0.503	1
1024	Python	95.07 s	6.9	88.54	0.023	1
1536	C	19.04 s	0.5	5.25	0.381	1
1536	Java	21.62 s	0.3	5.97	0.335	1
2048	C	48.46 s	0.3	5.64	0.355	1
2048	Java	45.19 s	0.9	5.26	0.380	1

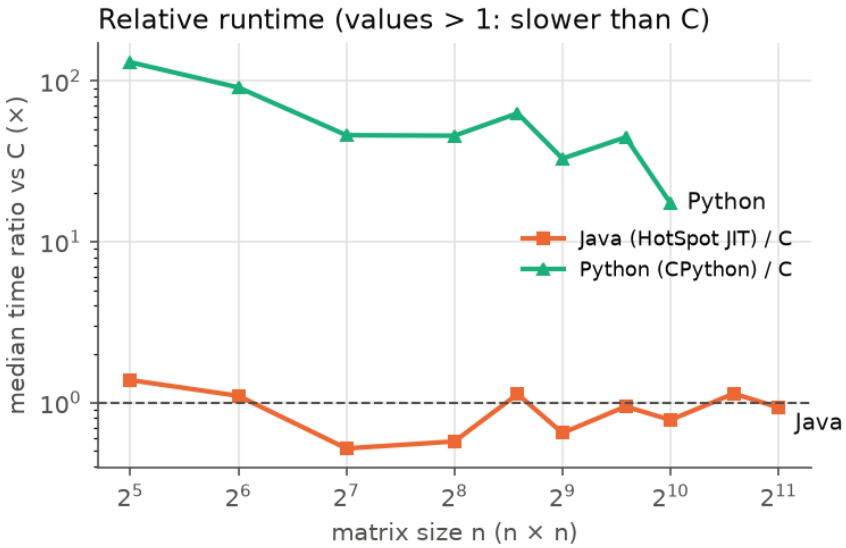


Figure 4: Median runtime relative to C at the same n.

4.3 Memory

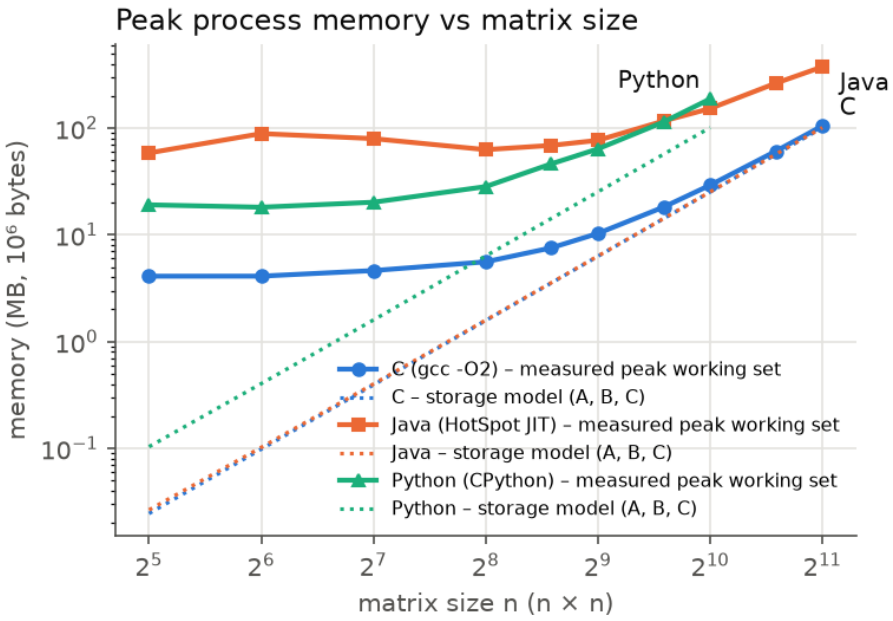


Figure 5: Peak working set per process (solid) and analytical storage of A, B, C (dotted; the C and Java models almost coincide).

Table 4: Peak working set of the whole process (psutil) versus the storage model of the three matrices. $1\,\text{MB} = 10^6$ bytes.

	Peak working set (MB)			Storage model (MB)		
n	C	Java	Python	C	Java	Python
32	4.1	58.4	19.1	0.0	0.0	0.1
64	4.1	88.6	18.1	0.1	0.1	0.4
128	4.6	79.5	20.2	0.4	0.4	1.6
256	5.6	62.7	28.3	1.6	1.6	6.3
384	7.5	68.7	46.2	3.5	3.6	14.2
512	10.3	76.6	63.7	6.3	6.3	25.3
768	18.2	116.2	113.7	14.2	14.2	56.8
1024	29.2	152.1	187.2	25.2	25.2	100.8
1536	60.6	262.5	–	56.6	56.7	–
2048	104.7	373.8	–	100.7	100.8	–

For Java, the managed heap in use after the last repetition grew from 18–45 MB at small n to 176 MB at $n = 2048$, while the committed heap stayed at its initial 260 MB (see `results/raw/runs.csv`).

4.4 Limits

- **C and Java** completed every size up to the largest attempted, $n = 2048$ (median 48.5 s and 45.2 s).
- **Python**'s largest completed size was $n = 1024$ (median 95.1 s); at $n = 1536$ no multiplication finished within 300 s and the run was stopped (peak working set 292 MB at that point), so $n = 2048$ was not attempted.
- **Memory:** no configuration came close to the 4 GB memory budget.

4.5 Replication and supplementary experiments

Replication

After the main run, mid-size configurations were run again in a separate session with the same protocol (Table 5). Within the main session the relative IQR was below 10 % for 21 of the 28 configurations (maximum 32 %, Java at $n = 768$), but medians differed between sessions by up to 58 %. The ordering of the languages was the same in both sessions at every replicated size.

Table 5: Median time in the main run and in a later replication session (`results/replication/`).

Language	n	Main run	Replication	Change
C	128	3.37 ms	2.92 ms	-13%
C	256	37.96 ms	36.49 ms	-4%
C	384	87.92 ms	72.99 ms	-17%
C	512	432.93 ms	683.97 ms	+58%
C	768	1.29 s	1.17 s	-9%
C	1024	5.48 s	5.46 s	-1%
Java	128	1.75 ms	1.40 ms	-20%
Java	256	21.74 ms	15.60 ms	-28%
Java	384	99.18 ms	81.45 ms	-18%
Java	512	279.73 ms	171.97 ms	-39%
Java	768	1.22 s	649.18 ms	-47%
Java	1024	4.27 s	3.76 s	-12%
Python	512	14.23 s	9.31 s	-35%
Python	768	57.52 s	42.12 s	-27%

Row padding in C (layout hypothesis)

To test whether cache conflicts caused by the power-of-two row stride explain C's behaviour, `c/supplementary/matmul_padded.c` runs the *same* kernel with each row stored $8n + 16$ bytes apart (two unused doubles per row, mimicking the Java header offset). The two layouts were measured alternately in one process to cancel drift. The checksums of both layouts were identical at every size.

Table 6: Supplementary (not part of the controlled comparison): same C kernel, contiguous rows versus rows padded by 16 bytes; 5 alternating repetitions, median.

n	C, pad 0 (row stride $8n$ B)	C, pad 2 ($8n + 16$ B)	Speed-up	Java (main run)
128	3.06 ms	1.34 ms	2.3×	1.75 ms
256	34.12 ms	12.02 ms	2.8×	21.74 ms
384	77.16 ms	41.47 ms	1.9×	99.18 ms
512	681.87 ms	101.70 ms	6.7×	279.73 ms
768	1.17 s	350.05 ms	3.4×	1.22 s
1024	5.96 s	867.40 ms	6.9×	4.27 s

GCC flags

The same C source was also built with other flag sets (Table 7). Loop unrolling and `-O3` did not make the kernel faster than `-O2`.

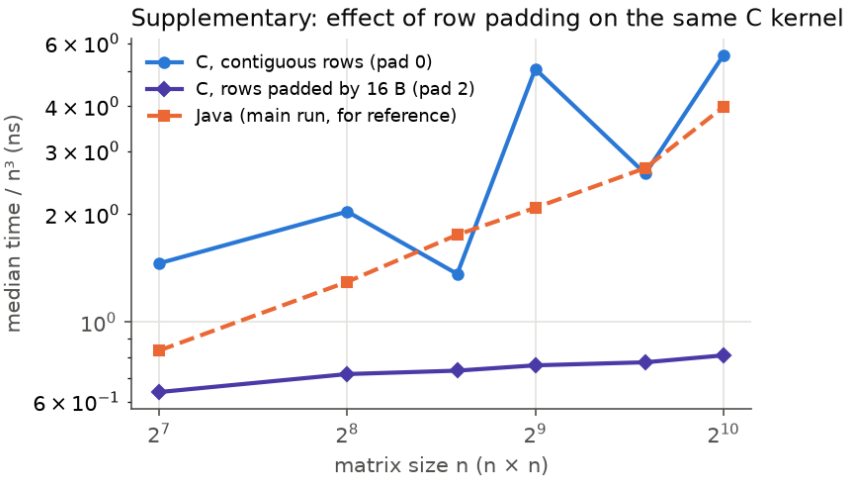


Figure 6: Supplementary: time per inner iteration of the same C kernel with and without row padding.

Table 7: Supplementary: median time of the unmodified C kernel with different GCC flags (separate session; variants run one after another, not interleaved).

GCC flags	$n = 256$	$n = 512$	$n = 1024$
O2 (main study)	32.98 ms	731.89 ms	5.51 s
O2 -funroll-loops	42.08 ms	739.22 ms	5.89 s
O3	36.23 ms	700.88 ms	5.50 s
O0	51.60 ms	459.98 ms	3.85 s

5 Discussion

Growth and the cubic model (H1)

The cubic model describes Python well: its cost per inner iteration varied only between 74 and 127 ns over a 32-fold range of n (Fig. 3) and the fitted exponent is 3.14. For C and Java, the exponent is clearly above 3 (3.50 and 3.73), so H1 is only partly supported. The algorithm still performs exactly n^3 iterations; what grows is the *cost per iteration*, from 0.6–0.95 ns at $n \leq 64$ to 4–6 ns at $n \geq 1024$.

At $n = 32$ all three matrices (24 KB) fit in L1. The C value there, 0.63 ns, matches roughly 3 cycles per iteration at the CPU's 4.5 GHz boost clock, which is consistent with the compiled loop (`movsd`, `mulsd`, `addsd`; checked with `gcc -S`) being limited by the latency of the floating-point additions chained through `s` [9, 8]. As n grows, each column walk of B touches n different cache lines, and the per-iteration cost rises when these no longer fit in L1, L2 and then L3.

The fitted exponent therefore mixes the algorithmic n^3 with a size-dependent memory cost. It should not be read as a property of the algorithm, and it depends on the size range.

Interpreter cost (H2)

Python was 17–130× slower than C (Fig. 4), which supports H2. Each Python inner iteration executes several bytecode instructions: two subscriptions of nested lists, a multiplication and an addition that each allocate a new `float` object, and loop control. This costs roughly 75–130 ns, compared with a few nanoseconds of memory stall in C.

Because the interpreter overhead dominates, Python is the least sensitive to cache effects and its curve is the closest to a pure n^3 law. Its ratio to C *falls* with n only because C becomes slower per iteration, not because Python becomes faster.

C versus Java: a layout effect, not a language ranking

Contrary to H2, Java was faster than C at most sizes from 128 to 1024. This held in both sessions, while C was faster at $n = 384$ in both (Tables 3 and 5). The C curve is irregular: per iteration, $n = 384$ and $n = 768$ are cheaper than the power-of-two sizes next to them (Fig. 3).

This pattern points to conflict misses. In C, consecutive rows of B are exactly $8n$ bytes apart, so for $n = 512$ the elements of a column are 4 KB apart and map to a single L1 set (64 sets of 64-byte lines). They also map to only 16 of the 1024 L2 sets, so only $16 \times 8 = 128$ of the 512 lines of one column can stay in L2.

The supplementary padding experiment supports this explanation. Shifting every row by 16 bytes, with no change to the arithmetic, made the same C code 1.9–6.9× faster, with the largest gains at $n = 512$ and $n = 1024$ (Table 6). Padded C was then faster than Java at every tested size and had an almost flat cost of 0.64–0.81 ns per iteration (Fig. 6). Java's separately allocated rows carry a 16-byte header, so its rows are normally not a power of two apart, which is a plausible reason why Java avoids most of the penalty.

We did not inspect the JIT-generated code or the actual heap addresses, so this explanation of Java's advantage is a supported hypothesis, not a proof. Unrolling (`-funroll-loops`) and `-O3` did not speed up C (Table 7), so instruction scheduling is unlikely to explain the gap. That table also contains one unexplained observation: `-O0` was faster than `-O2` at $n = 512$ and 1024. Explaining it would require hardware performance counters, which were not used.

Memory (H3)

Peak memory shows two regimes (Fig. 5). For small n it is the runtime baseline: about 4 MB for the C process, 18 MB for CPython and 58–90 MB for the JVM (heap, JIT, class data). For large n it follows n^2.

C's peak working set is the storage model plus about 4 MB (104.7 MB measured versus 100.7 MB modelled at $n = 2048$). Python's storage model is four times C's (32 instead of 8 bytes per element), which supports H3. Its measured peak (187 MB at $n = 1024$) exceeds the model because the interpreter baseline is added and because the previous result matrix remains referenced while the next one is computed in the repetition loop.

The JVM's peak (374 MB at $n = 2048$) is well above both its storage model (101 MB) and the heap in use (176 MB). G1 lets garbage from earlier repetitions accumulate before collecting it, and the JVM does not return committed memory to the OS immediately. The JVM's footprint is therefore governed by GC policy and heap settings (`-Xmx4g`) rather than by the matrices alone. Memory never limited any run; time was the binding constraint for Python.

Limitations

(i) One laptop with a "Balanced" power plan and dynamic frequency scaling. Medians varied by up to 58 % between sessions (Table 5), much more than the within-session IQR. Comparisons are reliable for ratios of several times (Python versus C/Java, padded versus unpadded C), but small differences between C and Java, such as at $n = 384$, are within this drift. Interleaving languages within a session, fixing the CPU frequency and running more sessions would reduce it.

(ii) Five repetitions per configuration, in one process each. Between-process (JIT) variance for Java was only sampled through the replication.

(iii) The peak working set is sampled every 20 ms. It is an OS high-water mark, so short peaks are not lost, but it includes runtime overheads and is not comparable to the Java heap metric.

(iv) The explanations of cache behaviour rely on layout arithmetic and the padding experiment, not on hardware counter measurements.

(v) Results hold for the i-j-k order and binary64. Other loop orders, which Assignment 2 will study, change the access pattern of B completely.

6 Conclusions

Key findings

On this machine and with these settings:

- The same $O(n^3)$ triple loop gave **bitwise-identical results** in C, Java and Python.
- CPython was **one to two orders of magnitude slower** than C and Java (17–130× relative to C) and could not complete $n = 1536$ within 300 s.
- C and Java differed by **less than a factor of two**, in both directions.
- Python's runtime followed the **cubic model** closely because interpretation dominates each iteration. C's and Java's runtime grew **faster than** n^3 over the tested range because the memory cost per iteration increases as the matrices leave the caches.
- The comparison between C and Java depended more on **memory layout** than on the language: a 16-byte row padding made the same C code up to 6.9× faster.
- Memory grew as n^2 once runtime baselines were exceeded, with Python using about four times C's storage per element and the JVM adding a large, GC-dependent overhead.

Open questions

A hardware-counter confirmation of the conflict-miss explanation, the reason for the fast `-O0` build, and how much of the session-to-session variation comes from frequency scaling. The validated implementations, inputs and scripts are reused as the baseline in Assignment 2.

Reproducibility materials

Repository: `https://github.com/amadoartiiles/matrix-multiplication-studies`. It contains:

- source code: `c/`, `java/`, `python/`;
- configuration: `config/benchmark_config.json`;
- input generation (`scripts/generate_inputs.py`) and the build, benchmark, validation and analysis scripts;
- all raw measurements: `results/raw/`, `results/replication/`, `results/supplementary/`;
- a README with exact build and run commands and dependency versions.

Use of generative AI

Claude (Anthropic) was used to help write the implementations, the benchmarking and analysis scripts, and a draft of this report. All numbers in this report come from running the code on the machine described above; the raw data are included.

References

[1] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Baltimore, MD, USA: Johns Hopkins Univ. Press, 2013.

[2] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[3] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, 2019.

[4] J. Darcy, "JEP 306: Restore Always-Strict Floating-Point Semantics," OpenJDK, 2021. [Online]. Available: https://openjdk.org/jeps/306

[5] M. S. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. ASPLOS-IV*, 1991, pp. 63–74.

[6] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., 2007. [Online]. Available: https://people.freebsd.org/~lstewart/articles/cpumemory.pdf

[7] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[8] Advanced Micro Devices, "Software Optimization Guide for AMD Family 19h Processors," Pub. no. 56665, 2020.

[9] A. Fog, "Instruction tables: Lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD and VIA CPUs," Technical University of Denmark. [Online]. Available: https://www.agner.org/optimize/

[10] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. OOPSLA*, 2007, pp. 57–76.

[11] G. Rodola, "psutil documentation," 2026. [Online]. Available: https://psutil.readthedocs.io/